TriaVet: Diccionario de Predicados Actualizado.

Módulo de Explicabilidad «¿Por qué?» y Módulo de Recomendación «¿Qué falta?»

Entregable 2 · Puente Python–PySwip y motor de explicabilidad («caja blanca»)

Presentado por:

María José Ospina Polania

Arlen José Perdomo Rojas

Presentado a:

Juan José Ramos Diaz

Fundación Escuela Tecnológica – FET

Facultad (Ingeniería)

Programa (ingeniería de Software)

Neiva - Huila

2026

Diccionario de predicados — TriaVet

Documenta los predicados de `base_conocimiento.pl` y cómo se invocan desde Python (`motor.py`).

Convención de modos de instanciación

Notación	Significado
`+Arg`	Entrada: debe llegar instanciado (enviado desde Python o desde la regla que llama).
`-Arg`	Salida: llega libre y el predicado lo instancia (se devuelve a Python).
`?Arg`	Entrada o salida: funciona instanciado o libre.

Un **caso** se representa internamente como `caso(Especie, EdadMeses, Sexo, Hallazgos)`, donde `Hallazgos` es la lista de signos y antecedentes válidos, ordenada y sin duplicados.

> **Actualización del Entregable 2.** Las secciones 10 a 14 documentan los predicados nuevos del **módulo de explicabilidad «¿Por qué?»** (Sección 8 del `.pl`), del **módulo de recomendación «¿Qué falta?»** (Sección 9) y de la **API para Python** del Entregable 2 (Sección 10). La sección 1 incluye los nuevos predicados públicos, y las secciones 5 a 7 y 9 marcan con **(E2)** los predicados de validación, de listas y los hechos agregados en las secciones anteriores.
>
> - 10. API para Python del Entregable 2
> - 11. Módulo «¿Por qué?»: metaintérprete de demostración y recolección de la traza
> - 12. Módulo «¿Por qué?»: refutación y análisis de hipótesis descartadas
> - 13. Módulo «¿Qué falta?»: recomendación y desempate
> - 14. Hechos nuevos, corte y negación en los módulos del Entregable 2

1. Predicados públicos invocados desde Python

Firma	Modo de instanciación	Descripción funcional	Invocación desde Python
`analizar_caso/10`	`analizar_caso(+Especie, +EdadMeses, +Sexo, +Entrada, -Validos, -Ignorados, -Alarmas, -Diagnosticos, -Prioridad, -Recomendacion)`	Consulta principal. Valida los datos del paciente, filtra y deduplica la lista de hallazgos, infiere los diagnósticos, calcula severidad, certeza, sustento y agravantes de cada uno, los ordena de mayor a menor urgencia y determina la prioridad global y su recomendación. Nunca falla: si los datos no son válidos responde `Prioridad = datos_invalidos`. Cada elemento de `Diagnosticos` es `[Id, Nombre, Severidad, Certeza, Indicacion, Sustento, Agravantes]`.	`MotorTriaVet.analizar_caso(especie, edad, sexo, hallazgos)` → `ResultadoCaso`
`diagnosticar_todos/5`	`diagnosticar_todos(+Especie, +EdadMeses, +Sexo, +Entrada, -Diagnosticos)`	Predicado principal de **recolección**: devuelve en una sola lista, ordenada y sin duplicados (`setof/3`), todos los diagnósticos posibles del caso. Sin coincidencias devuelve `[sin_diagnostico_concluyente]`; con datos inválidos devuelve `[]`.	`MotorTriaVet.diagnosticar_todos(...)` → `list[str]`
`catalogo_hallazgos/2`	`catalogo_hallazgos(?Grupo, -Hallazgos)`	Con `setof/3` agrupa los hallazgos válidos por sistema o tipo de antecedente. Con `Grupo` libre da una solución por grupo.	`MotorTriaVet.catalogo()` (menú de `consola.py` y selector de `main.py`)
`nombre_grupo/2`	`nombre_grupo(?Grupo, ?Nombre)`	Hecho: nombre legible de cada grupo de hallazgos.	`MotorTriaVet.catalogo()`
`nombre_diagnostico/2`	`nombre_diagnostico(?Diagnostico, ?Nombre)`	Hecho: nombre legible de cada diagnóstico.	`MotorTriaVet.nombre_diagnostico(id)`
(E2) `explicar_caso/5`	`explicar_caso(+Especie, +EdadMeses, +Sexo, +Entrada, -Explicaciones)`	Módulo **«¿Por qué?»**: traza de cada conclusión (árbol de inferencia, reglas disparadas, hechos verificados, justificación de severidad y certeza). Ver la sección 10.	`MotorTriaVet.explicar(...)` → `list[Explicacion]`
(E2) `hipotesis_descartadas/5`	`hipotesis_descartadas(+Especie, +EdadMeses, +Sexo, +Entrada, -Descartadas)`	Módulo **«¿Por qué?»**: hipótesis que comparten hallazgos con el caso pero no se concluyeron, con su motivo y su árbol de refutación.	`MotorTriaVet.descartadas(...)` → `list[HipotesisDescartada]`

Firma	Modo de instanciación	Descripción funcional	Invocación desde Python
(E2) `que_falta/8`	`que_falta(+Especie, +EdadMeses, +Sexo, +Entrada, +Negados, -Estado, -Competidoras, -Sugerencias)`	Módulo **«¿Qué falta?»**: estado de la evidencia, hipótesis en competencia y preguntas ordenadas por peso.	`MotorTriaVet.que_falta(...) → Recomendacion`
(E2) `pregunta_desempate/6`	`pregunta_desempate(+Especie, +EdadMeses, +Sexo, +Entrada, +Negados, -Pregunta)`	**Consulta de desempate**: la pregunta de mayor peso.	`MotorTriaVet.pregunta_desempate(...) → Sugerencia | None`
(E2) `nombre_hallazgo/2`	`nombre_hallazgo(?Hallazgo, ?Nombre)`	Hecho: nombre legible (con tildes) de cada uno de los 53 hallazgos.	`MotorTriaVet.nombres_hallazgos()`
`especie_paciente/2`, `sexo_paciente/2`	`(?Valor, ?Nombre)`	Hechos: valores admitidos para el paciente.	`MotorTriaVet.opciones_paciente()` (opciones del formulario)

Ejemplo de consulta que arma `motor.py`:

```
?- analizar_caso('perro', 5, 'macho', ['vomito', 'diarrea_con_sangre', 'anorexia'],
                 Validos, Ignorados, Alarmas, Diagnosticos, Prioridad, Recomendacion).
Prioridad = critico,
Diagnosticos = [[parvovirosis, 'Parvovirosis canina', critico, 80, '...',
                 [diarrea_con_sangre, vomito, anorexia], [paciente_vulnerable]]], ...
```

2. Recolección, severidad y prioridad (Secciones 6 y 7)

Firma	Modo de instanciación	Descripción funcional
`preparar_caso/6`	`preparar_caso(+Especie, +Edad, +Sexo, +Entrada, -Caso, -Ignorados)`	Verifica que `Entrada` sea una lista, la normaliza y construye `caso/4`. Falla si los datos del paciente no son válidos.
`conclusiones/2`	`conclusiones(+Caso, -Diagnosticos)`	`setof/3` sobre `diagnostico_posible/2`. Con corte y valor por defecto: lista vacía si no hay hallazgos y `[sin_diagnostico_concluyente]` si ningún perfil se cumple.
`diagnosticos_detallados/2`	`diagnosticos_detallados(+Caso, -Detalles)`	Detalla cada conclusión con `findall/3` y la ordena por severidad y certeza.

Firma	Modo de instanciación	Descripción funcional
detallar/3	detallar(+Caso, +Diagnostico, -Detalle)	Construye [Id, Nombre, Severidad, Certeza, Indicacion, Sustento, Agravantes].
indicacion_para/3	indicacion_para(+Caso, +Diagnostico, -Texto)	Indicación del diagnóstico. Con corte: si no hay diagnóstico pero sí signos de alarma, usa la indicación de urgencias.
ordenar_por_prioridad/2	ordenar_por_prioridad(+Detalles, -Ordenados)	Ordena de forma descendente con sort/4 según la clave peso_nivel * 1000 + certeza.
clave_orden/2	clave_orden(+Detalle, -Clave)	Calcula la clave numérica de ordenamiento.
severidad/3	severidad(+Caso, +Diagnostico, -Nivel)	Con un signo de alarma el nivel es critico (cláusula con corte). Si no, toma el nivel base y lo sube un nivel por cada agravante.
nivel_base/3	nivel_base(+Caso, +Diagnostico, -Nivel)	Para intoxicacion usa la toxicidad de la sustancia; para el resto, severidad_base/2.
severidad_intoxicacion/2	severidad_intoxicacion(+Caso, -Nivel)	El nivel más alto entre las sustancias ingeridas (findall/3 + max_member/2).
toxicidad_para/3	toxicidad_para(+Sustancia, +Especie, -Nivel)	Nivel registrado para esa sustancia y especie. Si no hay registro, aplica el valor por defecto (alto).
agravantes/3	agravantes(+Caso, +Diagnostico, -Lista)	Factores que agravan un cuadro agudo. Para cuadros crónicos devuelve [] (con corte).
agravante/2	agravante(+Caso, ?Agravante)	paciente_vulnerable, deshidratacion o compromiso_neurologico.
subir_niveles/3	subir_niveles(+Nivel, +K, -NivelFinal)	Sube K niveles en la escala; critico es el tope.
certeza/3	certeza(+Caso, +Diagnostico, -Porcentaje)	Factor de certeza: base + 0,10 por cada signo de apoyo presente, con tope de 0,95, expresado en porcentaje.
sustento/3	sustento(+Caso, +Diagnostico, -Hallazgos)	Hallazgos del caso que respaldan el diagnóstico; sirve de explicación.
perfil_presente/3	perfil_presente(+Diagnostico, +Hallazgos, -Presentes)	Intersección entre el perfil del diagnóstico y los hallazgos.
prioridad_caso/3	prioridad_caso(+Caso, +Detalles, -Prioridad)	Precedencia con cortes: sin hallazgos da indeterminado; un signo de alarma da critico; en otro caso, el nivel máximo de los diagnósticos.
nivel_maximo/2	nivel_maximo(+Detalles, -Nivel)	Recorre la lista de detalles y devuelve la severidad mayor.
mayor_nivel/3	mayor_nivel(+NivelA, +NivelB, -Mayor)	Compara dos niveles por su peso numérico.

Firma	Modo de instanciación	Descripción funcional
`entrada_como_lista/2`	`entrada_como_lista(+Entrada, -Lista)`	Garantiza que los hallazgos ignorados se devuelvan como lista.

3. Reglas de Nivel 3 — conclusión diagnóstica (Sección 5)

Firma	Modo de instanciación	Descripción funcional
`candidato/2`	`candidato(+Caso, ?Diagnostico)`	Hipótesis diagnóstica: una cláusula por diagnóstico que integra estados de Nivel 2 con `cumple_perfil/2`. Incluye descartes con `\+` (vacunación completa, cachorro, signos de alarma).
`excluido/2`	`excluido(+Caso, +Diagnostico)`	Verdadero si un diagnóstico dominante (`excluye/2`) también es candidato. Llama a `candidato/2`, no a `diagnostico_posible/2`, para evitar ciclos.
`diagnostico_posible/2`	`diagnostico_posible(+Caso, ?Diagnostico)`	`candidato/2` y `\+ excluido/2`. `Diagnostico` ya está instanciado cuando se evalúa la negación.

4. Reglas de Nivel 2 — inferencia intermedia (Sección 4)

Firma	Modo	Descripción funcional
`cuadro_digestivo/1`	`(+Caso)`	Hay al menos un indicador digestivo.
`deshidratacion/1`	`(+Caso)`	Hay dos o más indicadores de pérdida de líquidos.
`compromiso_neurologico/1`	`(+Caso)`	Hay convulsiones, temblores, desorientación o mioclonías.
`signos_de_shock/1`	`(+Caso)`	Depresión del estado general y perfusión deficiente.
`signo_de_alarma/1`	`(+Caso)`	Hay un signo de alarma directo o signos de shock.
`alarmas_presentes/2`	`(+Caso, -Alarmas)`	Lista los signos de alarma detectados.
`alarma_por_shock/2`	`(+Caso, -Lista)`	`[signos_de_shock]` o `[]` (corte y valor por defecto).
`abdomen_agudo/1`	`(+Caso)`	Dolor o distensión abdominal, junto con vómito persistente o arcadas.
`retencion_urinaria/1`	`(+Caso)`	Esfuerzo para orinar y ausencia de orina.
`exposicion_toxica/2`	`(+Caso, -Sustancia)`	Una solución por cada sustancia tóxica ingerida.
`hipertermia_probable/1`	`(+Caso)`	Exposición al calor y jadeo excesivo.
`sin_proteccion_vacunal/1`	`(+Caso)`	`\+` vacunación completa.
`susceptible_parvovirus/1`	`(+Caso)`	Sin protección vacunal, y además cachorro o con esquema incompleto.
`paciente_vulnerable/1`	`(+Caso)`	Cachorro, geriátrico, gestante o con enfermedad crónica previa.

Firma	Modo	Descripción funcional
`cumple_perfil/2`	`(+Caso, +Diagnostico)`	La especie es compatible, los requisitos son un subconjunto de los hallazgos y se alcanza el mínimo de signos de apoyo.

5. Reglas de Nivel 1 — validación preliminar (Sección 3)

Firma	Modo	Descripción funcional
`hallazgo_valido/1`	`(+Hallazgo)`	El átomo está registrado como signo o como antecedente.
`grupo_hallazgo/2`	`(?Hallazgo, ?Grupo)`	Relaciona cada hallazgo con su grupo.
`normalizar_entrada/3`	`(+Entrada, -Validos, -Ignorados)`	Ordena, deduplica (`sort/2`) y separa los hallazgos válidos de los ignorados.
`datos_paciente_validos/1`	`(+Caso)`	Especie y sexo conocidos, y edad entera entre 0 y 300 meses.
`etapa_vida/3`	`(+Especie, +EdadMeses, ?Etapa)`	`cachorro`, `geriatrico` o `adulto` (valor por defecto). La salida se unifica después del corte.
`etapa/2`	`(+Caso, ?Etapa)`	Etapa de vida del paciente del caso.
(E2) `entrada_segura/4`	`(+Especie, +Edad, +Sexo, +Entrada)`	Barrera de validación de lo que llega desde Python: `ground/1` (sin variables libres), especie y sexo como átomos en minúscula, edad entera y lista cerrada de hallazgos. La usa `preparar_caso/6`, de modo que protege a todos los predicados públicos.
(E2) `atomo_minuscula/1`	`(+Termino)`	El término es un átomo escrito en minúscula (`downcase_atom/2`).
(E2) `atomos_seguros/2`	`(+Entrada, -Lista)`	Valida la lista de hallazgos negados por el usuario; con corte y valor por defecto, cualquier entrada no válida se trata como `[]`.

6. Utilidades de procesamiento de listas (Sección 2)

Firma	Modo	Descripción funcional
`presente/2`	`(+Elemento, +Lista)`	Pertenencia determinista (`memberchk/2`).
`alguno_presente/2`	`(+Candidatos, +Lista)`	Al menos un candidato está en la lista.
`contiene_todos/2`	`(+Requeridos, +Lista)`	`Requeridos` es un subconjunto de `Lista`.
`contar_presentes/3`	`(+Lista, +Hallazgos, -N)`	Cuántos elementos de `Lista` aparecen en `Hallazgos`.

Firma	Modo	Descripción funcional
`interseccion/3`	`(+Lista, +Hallazgos, -Comunes)`	Filtra los elementos comunes conservando el orden.
`particionar/3`	`(+Lista, -Validos, -Invalidos)`	Separa los elementos válidos de los inválidos.
`pares_valores/2`	`(+Pares, -Valores)`	Extrae los valores de una lista `Clave-Valor`.
(E2) `diferencia/3`	`(+Lista, +Hallazgos, -Ausentes)`	Complemento de `interseccion/3`: elementos de `Lista` que no están en `Hallazgos`. Los módulos del Entregable 2 la usan para saber qué falta.

7. Hechos de la base de conocimiento (Sección 1)

Todos tienen aridad 2 o mayor y están agrupados de forma contigua por predicado.

Predicado	Cantidad	Contenido
`signo/2`	37	Signo clínico y sistema (digestivo, urinario, respiratorio, neurológico, general).
`antecedente/2`	16	Vacunación, condición, raza, exposiciones e ingestas.
`especie_afectada/2`	16	Especies en las que aplica cada diagnóstico.
`toxicidad/3`	13	Nivel de toxicidad por sustancia y especie.
`nombre_diagnostico/2`, `curso/2`, `requisito/2`, `apoyo/2`, `minimo_apoyo/2`, `certeza_base/2`, `severidad_base/2`	11 c/u	Perfil completo de los 10 diagnósticos más el valor por defecto.
`indicacion/2`	12	Qué hacer mientras se consigue atención (incluye la indicación de alarma sin diagnóstico).
`nombre_grupo/2`	9	Nombres legibles de los grupos de hallazgos.
`sustancia/2`	8	Ingesta reportada y sustancia tóxica.
`indicadores/2`	6	Listas que definen los estados intermedios.
`recomendacion/2`	6	Recomendación para cada nivel de prioridad.
`excluye/2`	5	Diagnóstico dominante y diagnóstico que excluye.
`umbral_edad/3`, `nivel_valor/2`, `siguiente_nivel/2`	4 c/u	Etapas de vida y escala de severidad.
`signo_alarma/2`	3	Signos que fijan la prioridad en crítico.

Predicado	Cantidad	Contenido
`especie_paciente/2`, `sexo_paciente/2`	2 c/u	Valores admitidos para el paciente.
(E2) `nombre_hallazgo/2`	53	Nombre legible de cada hallazgo (bloque 1.14).

Total del Entregable 1: 224 hechos de dominio en 23 predicados, y 98 cláusulas de reglas en 55 predicados.

Agregado en el Entregable 2: `nombre_hallazgo/2` (53 hechos) en la Sección 1; 13 predicados de hechos con 118 hechos en las Secciones 8 y 9 (ver la sección 14), y 105 predicados con 210 cláusulas de regla (con cuerpo) en las Secciones 8 a 10, más 4 predicados con 5 cláusulas de regla en las Secciones 2 y 3.

8. Uso del corte y de la negación por fallo

Mecanismo	Dónde	Para qué
Corte (`!`) entre cláusulas excluyentes	`contar_presentes/3`, `interseccion/3`, `particionar/3`, `hallazgo_valido/1`, `etapa_vida/3`, `severidad/3`, `nivel_base/3`, `subir_niveles/3`, `prioridad_caso/3`, `nivel_maximo/2`, `mayor_nivel/3`	Evita que el *backtracking* pruebe una cláusula alternativa que daría un resultado erróneo o repetido.
Corte con valor por defecto (*fallback*)	`etapa_vida/3` (adulto), `toxicidad_para/3`, `agravantes/3`, `alarma_por_shock/2`, `conclusiones/2`, `indicacion_para/3`, `diagnosticar_todos/5`, `analizar_caso/10`, `entrada_como_lista/2`	Garantiza una respuesta cuando ninguna condición específica se cumple, de modo que Python nunca recibe un fallo.
Corte de "basta uno"	`alguno_presente/2`, `signo_de_alarma/1`, `excluido/2`, `paciente_vulnerable/1`, `susceptible_parvovirus/1`	Detiene la búsqueda al encontrar la primera evidencia suficiente.
Negación por fallo (`\+`)	`sin_proteccion_vacunal/1`	Descarta parvovirosis y moquillo si se reporta vacunación completa.
	`candidato(_, enfermedad_renal_cronica)`	Descarta la enfermedad renal crónica en cachorros.
	`candidato(_, gastroenteritis_aguda)`	Descarta la gastroenteritis inespecífica si hay signos de alarma.
	`diagnostico_posible/2`	Descarta un diagnóstico si otro dominante lo excluye (p. ej. obstrucción uretral → cistitis).

En todas las negaciones, el caso (y el diagnóstico, cuando aplica) ya están instanciados antes de evaluar `\+`, por lo que la negación es segura.

9. Funciones de Python (`motor.py`)

Función / método	Qué hace
`MotorTriaVet(ruta_base)`	Carga `base_conocimiento.pl` con `Prolog.consult()`.
`MotorTriaVet.analizar_caso(...)`	Arma e invoca `analizar_caso/10` y convierte la respuesta en `ResultadoCaso`.
`MotorTriaVet.diagnosticar_todos(...)`	Invoca `diagnosticar_todos/5`.
`MotorTriaVet.catalogo()`	Invoca `nombre_grupo/2` y `catalogo_hallazgos/2`.
`MotorTriaVet.nombre_diagnostico(id)`	Invoca `nombre_diagnostico/2`.
`MotorTriaVet.consultar(consulta)`	Ejecuta cualquier consulta y devuelve las soluciones.
`termino_atomo()`, `termino_edad()`, `termino_lista()`	Convierten valores de Python en términos de Prolog entre comillas, para que un texto como `Fiebre` no se lea como una variable.
`formatear_resultado()`	Da formato de texto a la respuesta para la consola (solo presentación).
(E2) `MotorTriaVet.explicar(...)`	Invoca `explicar_caso/5` → `list[Explicacion]` con el árbol (`NodoTraza`), las reglas disparadas y los hechos verificados.
(E2) `MotorTriaVet.descartadas(...)`	Invoca `hipotesis_descartadas/5` → `list[HipotesisDescartada]`.
(E2) `MotorTriaVet.que_falta(..., negados)`	Invoca `que_falta/8` → `Recomendacion` (estado, competidoras y sugerencias).
(E2) `MotorTriaVet.pregunta_desempate(..., negados)`	Invoca `pregunta_desempate/6` → `Sugerencia` o `None`.
(E2) `MotorTriaVet.nombres_hallazgos()`, `opciones_paciente()`	Invocan `nombre_hallazgo/2`, `especie_paciente/2` y `sexo_paciente/2` para armar el formulario.
(E2) `MotorTriaVet.recargar()`	Vuelve a consultar el `.pl` (botón «Recargar base de conocimiento»).
(E2) `_HiloProlog`	Hilo único que ejecuta todas las consultas: SWI-Prolog asocia un motor a cada hilo y Streamlit atiende cada interacción en un hilo distinto.

`main.py` (Streamlit) solo captura y limpia las entradas, llama a estos métodos y dibuja los resultados (el árbol se convierte a Graphviz). `consola.py` conserva la consola del Entregable 1.

10. API para Python del Entregable 2 (Sección 10)

Los cuatro predicados validan la entrada con `preparar_caso/6` (que usa `entrada_segura/4`) y tienen una cláusula por defecto con corte, de modo que Python nunca recibe un fallo ni un error.

Firma	Modo de instanciación	Descripción funcional
`explicar_caso/5`	`explicar_caso(+Especie, +EdadMeses, +Sexo, +Entrada, -Explicaciones)` *det*	**«¿Por qué?»**: una explicación por cada conclusión, en el mismo orden de `diagnosticos_detallados/2`. Cada elemento es `[Id, Nombre, Arbol, Reglas, Hechos, Datos, Severidad, Certeza]`: el árbol de inferencia, la lista de reglas disparadas `[Nivel, Etiqueta, Descripcion]`, los hallazgos que unificaron, los datos del paciente validados, los pasos del cálculo de severidad y el cálculo de la certeza. Con datos no válidos devuelve `[]`.
`hipotesis_descartadas/5`	`hipotesis_descartadas(+Especie, +EdadMeses, +Sexo, +Entrada, -Descartadas)` *det*	**«¿Por qué?»**: hipótesis del catálogo que comparten al menos un hallazgo con el caso pero no se concluyeron, ordenadas por hallazgos compartidos. Cada una es `[Id, Nombre, Tipo, Resumen, Compartidos, Arbol]`, con `Tipo` ∈ {`exclusion`, `condicion_excluyente`, `dato_paciente`, `evidencia_insuficiente`, `regla`} y el árbol de refutación.
`que_falta/8`	`que_falta(+Especie, +EdadMeses, +Sexo, +Entrada, +Negados, -Estado, -Competidoras, -Sugerencias)` *det*	**«¿Qué falta?»**: `Estado = [Tipo, Activo, Mensaje]` (`Tipo` ∈ {`sin_hallazgos`, `insuficiente`, `empate`, `certeza_baja`, `definitivo`, `datos_invalidos`}; `Activo = si` cuando corresponde activar el módulo). `Competidoras`: `[Id, Nombre, Estado, Activacion, Gravedad, Coincidencias, Falta, Excluiria]`. `Sugerencias`: `[Tipo, Categoria, Hallazgos, Pregunta, Peso, Desempata, Motivos, Efecto, ComoVerificar]`, de mayor a menor peso. `Negados` son los hallazgos que el usuario respondió que no presenta.
`pregunta_desempate/6`	`pregunta_desempate(+Especie, +EdadMeses, +Sexo, +Entrada, +Negados, -Pregunta)` *semidet*	**Consulta de desempate**: la sugerencia de mayor peso de `que_falta/8`. Falla si no hay nada que preguntar.

Formato de un nodo del árbol (lo exporta `exportar_nodo/3`): `[Tipo, Nivel, Etiqueta, Descripcion, Hijos]`.

Tipo	Árbol	Significado
`regla`	demostración	Regla explicable que se disparó (`Nivel` 1, 2 o 3).
`verificacion`	demostración	Comprobación de lista (`alguno_presente/2`, `contiene_todos/2`, `contar_presentes/3`); sus hijos son los hallazgos que la cumplieron.

Tipo	Árbol	Significado
`hallazgo`	demostración	Hallazgo ingresado que unificó (hecho comprobado como verdadero).
`negacion`	demostración	`\+ Meta` que se cumplió (negación por fallo).
`dato`	demostración	Dato del paciente usado por una regla (especie, sexo o etapa).
`hecho`, `condicion`	demostración	Hecho de la base consultado o comparación verificada.
`no_regla`	refutación	Regla que falló; si tiene varias cláusulas, sus hijos son nodos `alternativa`.
`falta`, `falta_uno`, `apoyo_insuficiente`	refutación	Hallazgo obligatorio ausente, falta uno de varios o faltan signos de apoyo. Se corrigen agregando hallazgos.
`excluyente`	refutación	Una negación `\+` falló porque la condición sí se cumple; sus hijos son la evidencia.
`dato_incompatible`	refutación	La regla exige otra especie, sexo o etapa de vida.
`no_hecho`, `condicion_falsa`	refutación	Hecho inexistente o comparación falsa.

```
?- explicar_caso(perro, 4, macho, [vomito, diarrea_con_sangre, letargo, anorexia,
vacunacion_incompleta], E).
E = [[parvovirosis, 'Parvovirosis canina',
      [[regla, 3, 'diagnostico_posible(Caso, parvovirosis)', 'Conclusión: ...',
        [[regla, 3, 'candidato(Caso, parvovirosis)', ..., [...]],
         [negacion, 0, '\\+ excluido(Caso, parvovirosis)', 'Ningún diagnóstico dominante la
excluye', []]]]],
      [[3, 'diagnostico_posible(Caso, parvovirosis)', ...], [3, 'candidato(Caso, parvovirosis)',
...], ...],
      [anorexia, diarrea_con_sangre, letargo, vacunacion_incompleta, vomito],
      [[especie, perro], [edad_meses, 4], [sexo, macho], [etapa, cachorro]],
      ['Nivel base: ALTO (severidad_base/2).', 'Agravante: paciente vulnerable ...', 'Nivel
final: CRÍTICO.'],
      'certeza_base 0.60 + 0.10 × 4 signo(s) de apoyo = 1.00; se aplica el tope de 0.95 → 95
%.']].

?- pregunta_desempate(gato, 36, macho, [esfuerzo_para_orinar, vocalizacion_dolor], [], P).
P = [hallazgo, prueba, [ausencia_de_orina],
     '¿Al intentar orinar no sale orina o salen apenas unas gotas?', 24.1, si,
     ['Hallazgo obligatorio de «Obstrucción uretral felina» (gravedad CRÍTICO, activación 67
%)'],
     'Si lo presenta: se concluiría Obstrucción uretral felina; se descartaría Cistitis /
infección urinaria baja; la prioridad pasaría de MEDIO a CRÍTICO; el dictamen pasaría a ser
definitivo.',
     'Observe al animal varios minutos en el arenero ...'].
```

11. Módulo «¿Por qué?»: metaintérprete de demostración y recolección de la traza (Sección 8)

La traza no se escribe a mano: un **metaintérprete** vuelve a demostrar `diagnostico_posible(Caso, D)` con las mismas cláusulas de la base (`clause/2`) y registra cada paso. Por eso la explicación coincide exactamente con el razonamiento que produjo el diagnóstico.

Firma	Modo de instanciación	Descripción funcional
`demostrar/3`	`(+Caso, +Meta, -Nodos)` *nondet*	**Metaintérprete**. Conjunción: concatena las justificaciones. `\+ Meta`: negación por fallo, se registra como `negacion`. `once/1`: primera demostración. Regla explicable: se resuelve con `resolver/3` y se registra como `regla(Nivel, Meta, Hijos)`. Cualquier otra meta (hechos, utilidades de listas, aritmética) se ejecuta con `call/1` y se registra como hoja con `hojas/4`.
`resolver/3`	`(+Caso, +Meta, -Hijos)` *nondet*	Resuelve una regla explicable cláusula por cláusula con `clause/2`. Si el cuerpo tiene un corte, demuestra una vez la parte anterior y corta las demás cláusulas, igual que Prolog.
`separar_corte/3`	`(+Cuerpo, -Antes, -Despues)` *semidet*	Divide el cuerpo de una cláusula en lo anterior y lo posterior al primer `!`. Falla si no hay corte.
`explicable/2`	`(+Meta, -Nivel)` *semidet*	La meta corresponde a una regla que la traza debe abrir (`regla_explicable/3`). Usa `subsumes_term/2` para no instanciar la meta.
`descripcion_regla/2`	`(+Meta, -Descripcion)` *det*	Descripción del patrón más específico; texto vacío por defecto.
`hojas/4`	`(+Caso, +Patron, +Meta, -Nodos)` *det*	Convierte una meta primitiva ya demostrada en hojas: `presente/2`, `member/2` y `memberchk/2` sobre los hallazgos → `hallazgo`; utilidades de listas → `verificacion` con los hallazgos que la cumplieron; `etapa_vida/3` y `Caso = caso(...)` → `dato`; hechos → `hecho`; comparaciones → `condicion`. `Patron` es la meta antes de ejecutarse.
`hallazgos_a_nodos/2`	`(+Hallazgos, -Nodos)` *det*	Envuelve cada hallazgo en un nodo `hallazgo/1`.
`es_hecho_base/1`	`(+Meta)` *semidet*	La meta es un hecho de la base (predicado sin reglas, `predicate_property/2`).
`es_condicion/1`	`(+Meta)` *semidet*	Comparación aritmética o `memberchk/2`.

Firma	Modo de instanciación	Descripción funcional
`nodo_en/2`	`(+Nodos, -Nodo)` *nondet*	Recorre el árbol en preorden.
`hijos_de/2`	`(+Nodo, -Hijos)`	Hijos de los nodos `regla` y `verificacion`.
`reglas_disparadas/3`	`(+Caso, +Nodos, -Reglas)` *det*	**Reglas disparadas**: `findall/3` sobre el árbol y `list_to_set/2`; cada una es `[Nivel, Etiqueta, Descripcion]`.
`hechos_verificados/2`	`(+Nodos, -Hallazgos)` *det*	**Hechos comprobados como verdaderos**: hallazgos del usuario que unificaron (`sort/2`, sin duplicados).
`datos_del_paciente/2`	`(+Caso, -Datos)` *det*	Especie, edad, sexo y etapa de vida validados.
`texto_severidad/3`	`(+Caso, +Diagnostico, -Pasos)` *det*	Reconstruye paso a paso `severidad/3`: alarma (corte), o nivel base + un paso por agravante + nivel final.
`texto_nivel_base/3`, `textos_agravantes/3`, `nombre_alarma/2`	*det*	Textos de cada paso de la severidad.
`texto_certeza/3`, `componentes_certeza/6`	`(+Caso, +Diagnostico, -Texto)` *det*	Muestran el cálculo de `certeza/3`: base + 0.10 × signos de apoyo, con el tope de 0.95 cuando aplica.
`explicacion/3`	`(+Caso, +Diagnostico, -Explicacion)` *det*	Arma la explicación de una conclusión. Con corte, `sin_diagnostico_concluyente` se explica con la cláusula por defecto de `conclusiones/2`.
`etiqueta/3`, `texto_termino/2`, `abreviar/3`	`(+Caso, +Termino, -Texto)` *det*	Texto de un término para la traza; el caso completo se muestra como `Caso` y su lista de hallazgos como `H`.
`exportar_nodos/3`, `exportar_nodo/3`	`(+Caso, +Nodo, -Lista)` *det*	Convierten los nodos internos en `[Tipo, Nivel, Etiqueta, Descripcion, Hijos]`.
`descripcion_verificacion/2`, `lectura_negacion/3`	*det*	Lectura de las verificaciones y de las negaciones (`descripcion_negacion/3`).
`nombre_legible/2`, `enumerar/2`, `enumerar_hallazgos/2`, `texto_valor/2`	*det*	Textos legibles («a, b o c»).

12. Módulo «¿Por qué?»: refutación y análisis de hipótesis descartadas (Sección 8)

Para explicar por qué **no** se concluyó una hipótesis, un segundo metaintérprete refuta su regla `candidato/2`: indica qué meta falló y, si la meta es una regla, desciende en ella.

Firma	Modo de instanciación	Descripción funcional
`refutar/3`	`(+Caso, +Meta, -Fallos)` *det*	**Metaintérprete de refutación**. `\+ Meta` que falló → `excluyente(Meta, Evidencia)`; regla explicable → `no_regla(Nivel, Meta, Alternativas)` con una lista de fallos por cláusula; meta primitiva → `refutar_primitiva/3`.
`refutar_cuerpo/3`	`(+Caso, +Cuerpo, -Fallos)` *det*	Recorre la conjunción. Si una meta falla y **no comparte variables libres** con el resto del cuerpo (`independientes/2`), sigue revisando para informar todo lo que falta; si las comparte, se detiene.
`refutar_meta/5`, `continuar/3`, `refutar_resto/4`	*det*	Pasos de `refutar_cuerpo/3` (con corte y valor por defecto).
`independientes/2`	`(+MetaA, +MetaB)` *semidet*	Las metas no comparten variables libres (`term_variables/2`).
`refutar_clausulas/3`, `refutar_alternativas/3`, `parte_evaluada/2`	`(+Caso, +Meta, -Alternativas)` *det*	Refutan cada cláusula de una regla; respetan el corte: si la parte anterior a un `!` se cumplió, solo se refuta lo posterior y no se revisan más cláusulas.
`refutacion_especial/3`	`(+Caso, +Meta, -Fallos)` *semidet*	Lectura precisa de `cumple_perfil/2` (revisa especie, requisitos y apoyo por separado) y de `exposicion_toxica/2` (falta una ingesta).
`refutar_perfil/3`, `fallo_especie/3`, `fallo_apoyo/3`	`(+Caso, +Diagnostico, -Fallos)` *det*	Especie no admitida, requisitos faltantes (`diferencia/3`) y cuántos signos de apoyo faltan.
`refutar_primitiva/3`	`(+Caso, +Meta, -Fallos)` *det*	Hallazgo ausente (`falta`), ninguno de una lista (`falta_uno`), dato del paciente distinto (`dato_incompatible`), hecho inexistente o condición falsa.
`evidencia/3`	`(+Caso, +Meta, -Nodos)` *det*	Demostración de la meta negada que sí se cumple (por qué falló el `\+`).
`diagnostico_del_catalogo/1`	`(?Diagnostico)` *nondet*	Diagnósticos reales del catálogo (sin el valor por defecto).
`analizar_hipotesis/3`	`(+Caso, +Diagnostico, -Analisis)` *det*	`hipotesis(Estado, Coincidencias, Fallos, Requerimientos)` con `Estado` ∈ {`confirmada`, `excluida(Dominante)`, `parcial`, `bloqueada`}. Con corte, los estados se prueban en orden de precedencia. Lo comparten los dos módulos.
`coincidencias/3`, `hallazgos_del_perfil/2`, `disparador/2`	*det*	Hallazgos del caso que pertenecen al perfil (disparadores, requisitos y apoyo).
`reparable/1`, `todos_reparables/1`	`(+Fallo)` *semidet*	El fallo se corrige agregando hallazgos. Las reglas son monótonas en los hallazgos salvo en las negaciones, así que un `\+` fallido, la especie, el sexo o la edad **bloquean** la hipótesis.

Firma	Modo de instanciación	Descripción funcional
`estado_por_fallos/2`	`(+Fallos, -Estado)` *det*	`parcial` si todos los fallos son reparables; si no, `bloqueada`.
`requerimientos/2`, `requerimiento/2`, `requerimientos_de/3`	`(+Fallos, -Requerimientos)` *det*	Lo que habría que observar: `necesita(H)`, `uno_de(Lista)`, `k_de(K, Lista)`. En reglas con varias cláusulas elige la alternativa más barata.
`simplificar/2`, `descontar/5`	*det*	Quita los `uno_de/1` que ya cubre un hallazgo obligatorio y descuenta los obligatorios de los `k_de/2`.
`costo/2`, `costo_requerimiento/2`	`(+Requerimientos, -Costo)` *det*	Número de hallazgos que faltan.
`fallo_bloqueante/2`, `bloqueante/2`	`(+Fallos, -Fallo)` *semidet*	Primer fallo no reparable: el motivo del descarte.
`descartada/2`	`(+Caso, -Par)` *nondet*	Una hipótesis descartada con sus hallazgos compartidos.
`motivo_descarte/5`	`(+Estado, +Fallos, +Requerimientos, -Tipo, -Resumen)` *det*	Clasifica el descarte (exclusión, condición excluyente, dato del paciente o falta de evidencia crítica) y lo redacta.
`raiz_descarte/5`	*det*	Cuelga los fallos de la regla `candidato/2` que falló (la exclusión ya trae su raíz `excluido/2`).
`textos_requerimientos/2`, `texto_requerimiento/2`	*det*	Texto de cada requerimiento.

13. Módulo «¿Qué falta?»: recomendación y desempate (Sección 9)

Firma	Modo de instanciación	Descripción funcional
`estado_evidencia/3`	`(+Caso, +Detalles, -Estado)` *det*	`sin_hallazgos`, `insuficiente` (solo el valor por defecto), `empate(D1, D2, C1, C2)` (misma severidad y certezas a no más de `margen_empate` puntos; la severidad se comparte en las dos cabezas por unificación), `certeza_baja(D, C)` (por debajo de `umbral_confirmacion`) o `definitivo(D, C)`. Con corte, en orden de precedencia.
`describir_estado/4`	`(+Estado, -Tipo, -Activo, -Mensaje)` *det*	Decide si se activa el módulo (`Activo = si` en los estados insuficiente, empate y certeza baja) y redacta el mensaje.

Firma	Modo de instanciación	Descripción funcional
`hipotesis_en_competencia/3`	`(+Caso, +Negados, -Competidoras)` *det*	Hipótesis confirmadas y **parcialmente activas** (reparables y con al menos un hallazgo compartido), ordenadas por activación: `comp(D, Estado, Activacion, Coincidencias, Requerimientos)`.
`estado_competencia/4`	*det*	Confirmada, parcial o `refutada_usuario` (necesitaba un hallazgo que el usuario negó). Las excluidas y las bloqueadas no compiten.
`requerimiento_negado/2`	`(+Requerimientos, +Negados)` *semidet*	Algún requerimiento ya no puede cumplirse por las respuestas negativas.
`activacion/4`	`(+Estado, +Coincidencias, +Requerimientos, -Activacion)` *det*	Porcentaje del perfil cubierto: coincidencias / (coincidencias + costo de lo que falta); 100 para las confirmadas.
`activa/1`	`(+Competidora)` *semidet*	La competidora no fue refutada por el usuario.
`nivel_gravedad/3`, `gravedad/3`	`(+Caso, +Diagnostico, -Nivel)` *det*	Gravedad potencial para este paciente: el nivel base del diagnóstico (`nivel_base/3`; en la intoxicación, el de la sustancia ingerida) y su peso de 1 a 4. Con corte y valor por defecto, sin ingesta usa `severidad_base/2`.
`sugerencias/5`	`(+Caso, +Estado, +Negados, +Competidoras, -Sugerencias)` *det*	Pondera cada pregunta candidata y las ordena de mayor a menor peso (`sort/4`).
`foco/3`, `empatada/2`	`(+Estado, +Competidoras, -Foco)` *det*	En un empate solo se ponderan las dos hipótesis empatadas (el objetivo es separarlas); en los demás estados, todas las activas.
`pregunta_candidata/4`	`(+Caso, +Negados, +Activas, -Pregunta)` *nondet*	`hallazgo(H)` aún no ingresado ni negado, relevante para alguna hipótesis y no contradictorio (`contradice/2`); o `grupo(Opciones)` para un `uno_de/1` abierto.
`hallazgo_relevante/3`, `contradice/2`	*nondet* / *semidet*	Relevancia por requerimiento o por signo de apoyo; incompatibilidad con lo ya ingresado (`incompatibles/2`).
`rol/3`	`(+Competidora, +Pregunta, -Rol)` *semidet*	Papel principal de la pregunta: `requisito` (3), `alternativa` (3), `minimo` (2) o `refuerzo` (1). Con corte, de mayor a menor peso.

Firma	Modo de instanciación	Descripción funcional
`ponderar/6`	`(+Caso, +Pregunta, +Activas, -Peso, -Desempata, -Motivos)` *semidet*	**Peso = Σ peso_rol × gravedad × activación / 100**, multiplicado por `factor_desempate` si la pregunta desempata; redondeado a un decimal.
`desempata/3`, `factor/2`	`(+Activas, +Motivos, -Desempata)` *det*	`si` cuando hay al menos dos hipótesis en el foco y la pregunta tiene su rol más alto en una sola: confirmarla o descartarla separa esa hipótesis de las demás.
`efecto/3`	`(+Caso, +Pregunta, -Texto)` *det*	**Simulación «¿qué pasaría si…?»**: vuelve a inferir el caso con el hallazgo agregado.
`simular/3`	`(+Caso, +Hallazgo, -Cambio)` *det*	`cambio(Ganados, Perdidos, Certezas, PrioridadAntes, PrioridadDespues, Definitivo)`: diagnósticos que aparecen o desaparecen, certezas que suben, cambio de prioridad y si el dictamen pasa a ser definitivo o se rompe el empate.
`pasa_a_definitivo/3`, `redactar_efecto/3`, `partes_efecto/2`, `parte_diagnosticos/3`, `parte_prioridad/3`, `parte_definitivo/2`, `partes_o_sin_cambio/2`	*det*	Redacción del efecto (cortes con valor por defecto).
`texto_pregunta/2`	`(+Pregunta, -Texto)` *det*	Pregunta registrada (`pregunta/2`), pregunta de grupo (sustancias tóxicas, vómito o diarrea) o genérica.
`categoria/2`, `instruccion/2`	`(+Pregunta, -Valor)` *det*	`prueba` (si existe `como_verificar/2`), `sintoma` o `antecedente`, y la instrucción de la prueba.
`hallazgos_pregunta/2`, `tipo_pregunta/2`	*det*	Hallazgos y tipo (`hallazgo` o `grupo`) de una pregunta.
`exportar_sugerencia/3`, `texto_motivo/3`, `exportar_competidora/4`	*det*	Exportan las sugerencias y competidoras a listas para Python.

14. Hechos nuevos, corte y negación en los módulos del Entregable 2

Predicado	Hechos	Contenido
`nombre_hallazgo/2` (bloque 1.14)	53	Nombre legible de cada hallazgo.
`pregunta/2`	53	Cómo preguntarle al tutor por cada hallazgo.
`regla_explicable/3`	27	Reglas que la traza abre, con su nivel y su descripción.
`como_verificar/2`	8	Prueba sencilla que el tutor puede hacer (llenado capilar, pliegue cutáneo, temperatura…).

Predicado	Hechos	Contenido
`nombre_nivel/2`	5	Nombre de cada nivel de severidad.
`descripcion_negacion/3`	4	Lectura de cada `\+` cuando se cumple y cuando falla.
`peso_rol/2`, `nombre_rol/2`	4 c/u	Peso y nombre del papel de un hallazgo en una hipótesis.
`parametro/2`	3	`umbral_confirmacion` (80 %), `margen_empate` (5 puntos) y `factor_desempate` (3).
`nombre_agravante/2`	3	Nombre de cada agravante.
`incompatibles/2`	1	Hallazgos que no pueden presentarse a la vez (vacunación completa e incompleta).
`hijos_de/2`, `hallazgos_pregunta/2`, `tipo_pregunta/2`	2 c/u	Hechos auxiliares de estructura.

Mecanismo	Dónde	Para qué
Metaprogramación (`clause/2`, `call/1`, `subsumes_term/2`, `copy_term/2`)	`demostrar/3`, `resolver/3`, `refutar/3`, `refutar_clausulas/3`, `explicable/2`, `hojas/4`	La traza se obtiene de las propias reglas: si cambia la base, cambia la explicación.
Corte entre formas de meta	`demostrar/3`, `refutar/3`, `hojas/4`, `refutar_primitiva/3`, `exportar_nodo/3`, `texto_termino/2`	Cada forma (conjunción, negación, regla, primitiva) se atiende una sola vez.
Corte que **reproduce** el corte de la cláusula original	`resolver/3`, `refutar_alternativas/3`	La traza sigue exactamente el mismo camino que Prolog.
Corte con valor por defecto	`descripcion_regla/2`, `evidencia/3`, `estado_por_fallos/2`, `requerimiento/2`, `estado_evidencia/3`, `estado_competencia/4`, `activacion/4`, `desempata/3`, `factor/2`, `pasa_a_definitivo/3`, `motivo_descarte/5`, `atomos_seguros/2`, `explicar_caso/5`, `hipotesis_descartadas/5`, `que_falta/8`	Siempre hay respuesta; Python nunca recibe un fallo.
Negación por fallo (`\+`)	`demostrar/3` (registra `negacion`), `independientes/2`, `simplificar/2`, `pregunta_candidata/4`, `reparable/1` vía `fallo_bloqueante/2`	Siempre sobre términos instanciados: la meta negada ya está ligada al caso.
Recolección (`findall/3`, `sort/4`, `list_to_set/2`)	`reglas_disparadas/3`, `hechos_verificados/2`, `hipotesis_en_competencia/3`, `sugerencias/5`, `hipotesis_descartadas/5`	Listas sin duplicados y ordenadas por activación o por peso.
Validación con `ground/1`	`entrada_segura/4`, `atomos_seguros/2`	Ninguna variable libre enviada desde Python llega a las reglas.